JobPulse — Data Engineering Project Documentation

Automated ETL Pipeline Architecture, Apache Airflow DAG, and Analytics Dashboard

Project Domain: Data Engineering / ETL Workflow	**Orchestrator:** Apache Airflow (Dockerized)
Database: PostgreSQL 15	**Frontend & API:** Flask + Modern Animated Dashboard

1. AIM

The primary objective of the **JobPulse Data Engineering Project** is to design, develop, and deploy an automated, end-to-end Data Engineering **ETL (Extract, Transform, Load) Pipeline** orchestrated by **Apache Airflow**.

Key goals include:

- **Automated Data Ingestion:** Extract raw job market datasets from CSV files or live user uploads via an API endpoint.
- **Automated Workflow Orchestration:** Schedule and execute tasks in precise order using Apache Airflow Directed Acyclic Graphs (DAGs).
- **Data Transformation & Cleaning:** Standardize job titles, clean messy salary ranges, extract technology skills, and calculate salary midpoints using Python and Pandas.
- **Data Quality Validation:** Enforce schema validation rules (e.g., non-null checks, realistic numeric ranges) post-transformation before loading to database.
- **Relational Storage:** Persist raw data, cleaned records, skill relationships, aggregated analytics, and execution logs into PostgreSQL tables.
- **Interactive SaaS Web Dashboard:** Provide a sleek, animated, user-friendly interface for uploading raw datasets, executing the ETL pipeline in real-time, and viewing analytical charts.

2. THEORY

Modern enterprise data architectures rely on robust **Data Engineering Pipelines** to turn raw, unstructured, or semi-structured data into high-value analytics. Below are the key foundational concepts utilized in JobPulse:

A. ETL Architecture (Extract, Transform, Load)

ETL is the core standard in data engineering. *Extraction* retrieves data from source systems. *Transformation* cleanses, normalizes, and enriches raw records. *Loading* writes formatted data into target data stores (data warehouses or operational databases) for analytical querying.

B. Workflow Orchestration with Apache Airflow

Workflow orchestration coordinates complex multi-step data pipelines. **Apache Airflow** is an open-source platform programmatically configured using Python. It ensures tasks execute sequentially based on dependencies, handles retries on failures, logs task outcomes, and monitors execution status through a web console.

C. Directed Acyclic Graph (DAG)

A **DAG** is a collection of all tasks you want to run, organized in a way that reflects their relationships and dependencies. It is **Directed** because tasks execute in a specific direction (A → B → C), and **Acyclic** because loops are prohibited, preventing infinite execution deadlocks.

D. Containerization via Docker & Docker Compose

Docker packages applications and dependencies into isolated containers. Docker Compose coordinates multi-container services (Airflow Webserver, Airflow Scheduler, PostgreSQL Database, and Dashboard Application) ensuring identical environment configuration across development and production.

3. EXPLANATION (TOOLS & COMPONENTS BREAKDOWN)

Each component in the JobPulse architecture is selected for a specific data engineering role:

Technology / Tool	Role / Purpose in JobPulse	Implementation Detail
Apache Airflow	Pipeline Orchestrator & Task Scheduler	Schedules daily DAG runs, manages task dependencies, triggers Python scripts via PythonOperator, handles retries & logging.
Python & Pandas	Data Manipulation & Transformation Engine	Reads raw CSV data, performs text cleanup, standardizes job titles, computes salary statistics, parses skill keywords, and exports DataFrames.
PostgreSQL 15	Relational Data Warehouse & Storage	Houses structured relational tables: raw_jobs, transformed_jobs, job_skills, job_analytics, and pipeline_runs.
Flask Web Server	Backend Web API & Data Provider	Serves dashboard HTML templates, exposes REST endpoints (/api/upload, /api/trigger-pipeline, /api/metrics) to trigger ETL inline.
Modern HTML5/JS Frontend	Interactive UI & Analytical Visualization	Renders live charts (Chart.js), dataset preview tables, KPI counters, and CSV file upload modal with real-time status notifications.
Docker & Compose	Infrastructure Orchestration	Runs Airflow Postgres, Scheduler, Webserver, and Dashboard backend in isolated environment containers via docker-compose.yml.

Detailed Breakdown: What is DAG & Where is it Implemented?

- **File Location:** Implemented in dags/jobpulse_etl_dag.py under the DAG ID jobpulse_etl_pipeline.

• **Purpose:** Defines the exact workflow structure, task execution order, execution schedule (@daily), default parameters (retries=1, retry_delay=5 mins), and task callables.

• **Task Ordering (Dependency Chain):**

```
extract_task >> transform_task >> validate_task >> load_task >> analytics_task >> log_task
```

4. WORKFLOW (STEP-BY-STEP OVERVIEW)

The end-to-end execution flow of JobPulse operates seamlessly from user interaction to database visualization:

Step 1: Dataset Upload

User accesses the web dashboard at http://localhost:5000 and uploads a CSV file containing raw job postings (or uses the default seed dataset).

Step 2: API Request & Trigger

The dashboard backend endpoint /api/upload receives the file, saves it to data/raw_jobs.csv, and executes the ETL execution script inline.

Step 3: Airflow DAG Execution

Airflow (or local runner) executes the 6 sequential tasks defined in the DAG dependency graph.

Step 4: Data Processing & Cleaning

Raw CSV rows are parsed, salary values are extracted into numeric format, job categories are assigned, missing fields are handled, and technology skills are isolated.

Step 5: Quality Check

Automated validation asserts data integrity (verifies schema consistency, non-negative salary midpoints, and expected record counts).

Step 6: Storage in PostgreSQL

Validated records are inserted into PostgreSQL target tables (transformed_jobs, job_skills, and job_analytics).

Step 7: Real-Time Dashboard Update

The frontend dashboard automatically re-queries the PostgreSQL database via Flask REST APIs and updates analytical charts and counters without requiring a page refresh.

5. PIPELINE (DETAILED TASK BREAKDOWN)

The JobPulse pipeline consists of 6 sequential tasks executed by Apache Airflow Python Operators:

Task: 1. Extract Task (extract_csv_data) (Script: scripts/extract.py)

Reads the raw input file from data/raw_jobs.csv. Validates column headers (e.g., job_title, company, location, salary, skills). Inserts raw un-modified records directly into PostgreSQL table raw_jobs for auditability and lineage tracking.

Task: 2. Transform Task (transform_data) (Script: scripts/transform.py)

Applies Python and Pandas cleaning transformations:
• Normalizes job titles into standard categories (Data Engineer, Data Scientist, ML Engineer, Data Analyst).
• Cleans salary strings (e.g., '$120,000 - $150,000') into salary_min, salary_max, and calculates salary_midpoint.
• Handles missing values with sensible defaults ('Unknown Location', 'Not Specified').
• Standardizes experience levels (Entry, Mid, Senior, Lead).

Task: 3. Validate Task (validate_raw_data) (Script: scripts/validate.py)

Performs Data Quality Assurance on transformed records post-cleaning:
• Asserts total transformed record count matches raw extracted count.
• Checks that mandatory fields (job_title, company) contain zero null values.
• Verifies salary_midpoint contains positive non-negative numerical values.

Task: 4. Load Task (load_transformed_data) (Script: scripts/load.py)

Writes processed DataFrames into PostgreSQL relational database tables:
• Inserts cleaned jobs into transformed_jobs table.
• Extracts skill arrays, explodes individual skills, and populates job_skills mapping table for skill frequency analysis.

Task: 5. Analytics Task (compute_analytics) (Script: scripts/analytics.py)

Computes pre-aggregated business metrics and statistical rollups:
• Average, min, and max salary midpoints grouped by Job Role.
• Top 10 most in-demand technical skills across all job listings.
• Distribution of job postings across locations and remote work types.
• Stores pre-computed metric outputs into job_analytics table for instant dashboard retrieval.

Task: 6. Log Task (log_pipeline_run) (Script: scripts/jobpulse_etl_dag.py)

Logs execution status, total processed records count, execution timestamp, and completion state into pipeline_runs table for operational observability.

6. CONCLUSION

The **JobPulse Data Engineering Pipeline** successfully demonstrates a modern, enterprise-grade ETL workflow built on solid data engineering fundamentals.

Key Takeaways & Achievements:

- **Reliable Orchestration:** Apache Airflow DAGs guarantee strict sequential execution and error handling for complex data pipelines.
- **Robust Data Quality:** Post-transformation validation prevents bad data from corrupting downstream database tables and analytical reports.
- **Scalable Storage Architecture:** PostgreSQL table normalization separates raw audit logs, cleaned transactional data, exploded skill relations, and aggregated analytics metrics.

- **Seamless User Experience:** The interactive web dashboard allows users to upload custom datasets and immediately view transformed analytical insights with visually striking charts.

This project serves as a comprehensive, end-to-end practical demonstration of Data Engineering principles suitable for academic assessment and practical industry deployment.